Проектирование микропроцессорной системы на базе МК i8051

Описание программной части

Листов 29

Выполнил: Рязанцев И.В.
Группа: А-12-23
Вариант: 17

Проверил: Михалин С.Н.

Москва, 2026

Подп. и дата
Инв. № дубл.
Взам. инв. №
Подп. и дата
Инв. № подл.

АННОТАЦИЯ

В данном документе приведено описание программы «Программа управления объектом», предназначенной для управления объектом микропроцессорной системой (МПС) на базе микроконтроллера AT89S53: ввода данных с клавиатуры и от внешнего устройства, формирования управляющих сигналов Y_1 и Y_2 и вывода информации на семисегментный индикатор.

Для функционирования данной программы необходима МПС, собранная по схеме электрической принципиальной варианта 17 (пояснительная записка по аппаратной части).

Исходным языком программы «Программа управления объектом» является ассемблер микроконтроллеров семейства MCS-51.

Основными функциями программы являются: инициализация МПС, сканирование клавиатуры, работа с кольцевым буфером на 1774 байт, выработка строба Y_1 длительностью 361 мс, выработка сигнала Y_2 со стробом 2422 мкс, индикация на одном разряде.

Оформление программного документа «Описание программы» произведено по требованиям ЕСПД (ГОСТ 19.101-77, ГОСТ 19.103-77, ГОСТ 19.104-78, ГОСТ 19.105-78, ГОСТ 19.106-78, ГОСТ 19.402-78, ГОСТ 19.604-78).

СОДЕРЖАНИЕ

1 Общие сведения..... 4
2 Функциональное назначение..... 5
2.1 Классы решаемых задач..... 5
2.2 Назначение программы..... 5
2.3 Сведения о функциональных ограничениях на применение..... 5
3 Описание логической структуры..... 6
3.1 Алгоритм программы..... 6
3.2 Структура программы с описанием функций составных частей и связи между ними 19
3.3 Связи программы с другими программами..... 20
4 Входные данные..... 21
4.1 Клавиатура..... 21
4.2 Сигнал X2..... 21
5 Выходные данные..... 22
5.1 Характер и организация выходных данных..... 22
5.2 Формат, описание и способ кодирования выходных данных..... 22
ПЕРЕЧЕНЬ ПРИНЯТЫХ СОКРАЩЕНИЙ..... 24
ЛИСТ РЕГИСТРАЦИИ ИЗМЕНЕНИЙ..... 25
ПРИЛОЖЕНИЕ А. Листинги программ..... 26

1 Общие сведения

Обозначение и наименование программы — «Программа управления объектом», вариант 17, группа А-12-23.

Программа низкого уровня не требует для своей работы иных программ. Исходным языком программирования является ассемблер MCS-51 (синтаксис Intel ASM51, совместимый с Keil A51 и MCU 8051 IDE).

Среда разработки и инструментальные средства: интегрированная среда Keil µVision V5 (Keil Software, пакет PK51); ассемблер — A51; компоновщик — BL51.

2 Функциональное назначение

2.1 Классы решаемых задач

- инициализация МПС;
- сканирование клавиатуры — ввод сигнала X_1 (число, введённое с клавиатуры);
- формирование сигнала Y_1 (строб заданной длительности);
- ввод сигнала X_2 (сохранение данных в кольцевом буфере);
- формирование сигнала Y_2 и сопровождающего строба;
- вывод символа на индикатор.

2.2 Назначение программы

МПС принимает информацию $\{X\}$ об объекте управления от датчиков, вырабатывает управляющие воздействия $\{Y\}$ в соответствии с заданным алгоритмом управления и подаёт их на исполнительные механизмы. Программа разбита на подзадачи по функциональному признаку:

- инициализация МПС;
- обработка прерываний: опрос клавиатуры, запись в буфер, таймеры генераторов сигналов Y;
- определение кода нажатой клавиши;
- вывод данных на семисегментный индикатор;
- формирование сигнала Y_2;
- обеспечение работы кольцевого буфера (запись и чтение отсчётов X_2).

В соответствии с вариантом (табл. 1 ТЗ) разработан код трёх программ:

- программа 1 — обработчик прерывания клавиатуры (сканирование матрицы), код нажатой клавиши в аккумуляторе или 0FFh;
- программа 2 — запись в кольцевой буфер (данные через аккумулятор) с управлением флагами пустоты и переполнения;
- программа 3 — расчёт и выдача сигнала Y_2 со стробом длительностью T_2 (прерывание таймера).

2.3 Сведения о функциональных ограничениях на применение

В качестве основы для построения МПС выбран микроконтроллер семейства MCS-51 — AT89S53, тактовая частота 12 МГц (машинный цикл 1 мкс). Таймер 2 и сторожевой таймер не используются (совместимость с i8051).

Интерфейсы:

- входные данные X_2 — 8-разрядное целое без знака по независимой параллельной шине со стробом записи низкого уровня в двухпортовое ОЗУ, нерегулярно, до 4 отсчётов в секунду;
- входные данные X_1 — десятичная цифра 0–9 с матричной клавиатуры 3×4 (сканирование по столбцам);
- выходной сигнал Y_2 — 8-разрядный параллельный код в выходном регистре со стробом низкого уровня длительностью 2422 мкс на выводе P1.4;
- выходной сигнал Y_1 — строб низкого уровня длительностью 361 мс на выводе P1.3;
- индикация — один семисегментный индикатор (общий анод), время индикации символа не более T_3 = 3,190 с.

3 Описание логической структуры

3.1 Алгоритм программы

После сброса программа один раз выполняет инициализацию (рисунок 1): ставит указатель стека на 07h, включает дешифратор адреса сигналом CS_EN, приводит стробы Y1 и Y2 в пассивную единицу, опускает все столбцы клавиатуры, делает буфер пустым (голова и хвост — на первой ячейке, флаг пустоты взведён), настраивает оба таймера в 16-разрядный режим и разрешает прерывания INT0, INT1, Timer0 и Timer1. Дальше работает основной цикл (рисунок 2): пока цифра X1 не равна нулю и в буфере есть отсчёты, он забирает очередной отсчёт X2 и выдаёт новое значение Y2, а при X1 = 0 один раз выдаёт нулевое Y2.

Всё, что связано с внешними событиями, сделано в прерываниях. Нажатие клавиши даёт INT0: обработчик опрашивает матрицу (рисунок 3) и по коду клавиши запоминает X1, показывает символ, запускает строб Y1 или очищает буфер (рисунок 4). Строб X2 от внешнего устройства даёт INT1: обработчик вызывает процедуру записи, которая выполняет movx по адресу «головы» — правый порт буфера заносит в ячейку байт X2 с разъёма, — сдвигает «голову» и обновляет флаги (рисунки 5 и 6). Длительности не отсчитываются в циклах ожидания: Timer0 с тиком 1 мс заканчивает строб Y1 и гасит индикатор через T3 (рисунок 10), Timer1 одним отсчётом заканчивает строб Y2 (рисунок 13).

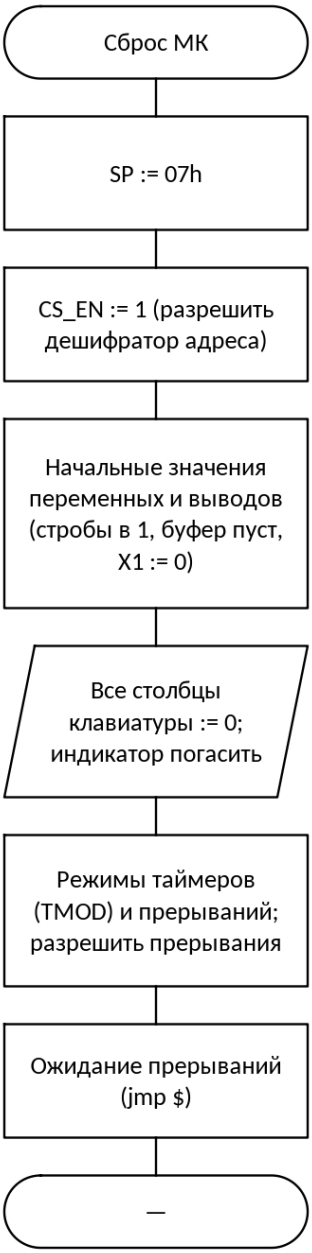


Рисунок 1 — Схема алгоритма: Инициализация МПС и основная программа

Схема «Инициализация МПС и основная программа» выполняется так: вход — «Сброс МК»; SP := 07h; CS_EN := 1 (разрешить дешифратор адреса); начальные значения переменных и выводов (стробы в 1, буфер пуст, X1 := 0); обмен с устройством: все столбцы клавиатуры := 0; индикатор погасить; режимы таймеров (TMOD) и прерываний; разрешить прерывания; ожидание прерываний (jmp $).

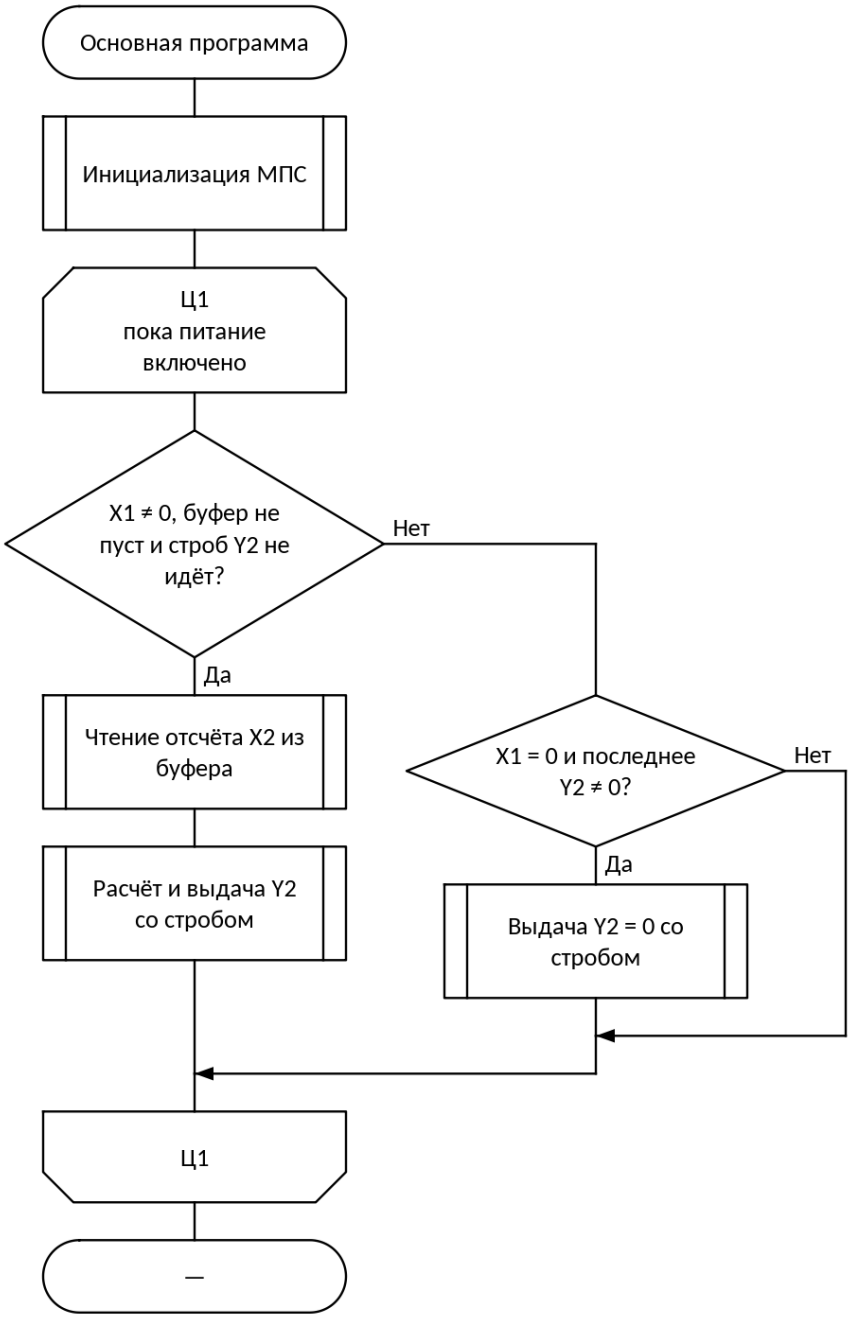


Рисунок 2 — Схема алгоритма: Основной цикл программы

Схема «Основной цикл программы» выполняется так: вход — «Основная программа»; вызывается подпрограмма: инициализация МПС; в цикле Ц1 (пока питание включено) повторяется: проверяется условие «X1 ≠ 0, буфер не пуст и строб Y2 не идёт»: если да — вызывается подпрограмма: чтение отсчёта X2 из буфера, вызывается подпрограмма: расчёт и выдача Y2 со стробом, иначе — проверяется условие «X1 = 0 и последнее Y2 ≠ 0»: если да — вызывается подпрограмма: выдача Y2 = 0 со стробом.

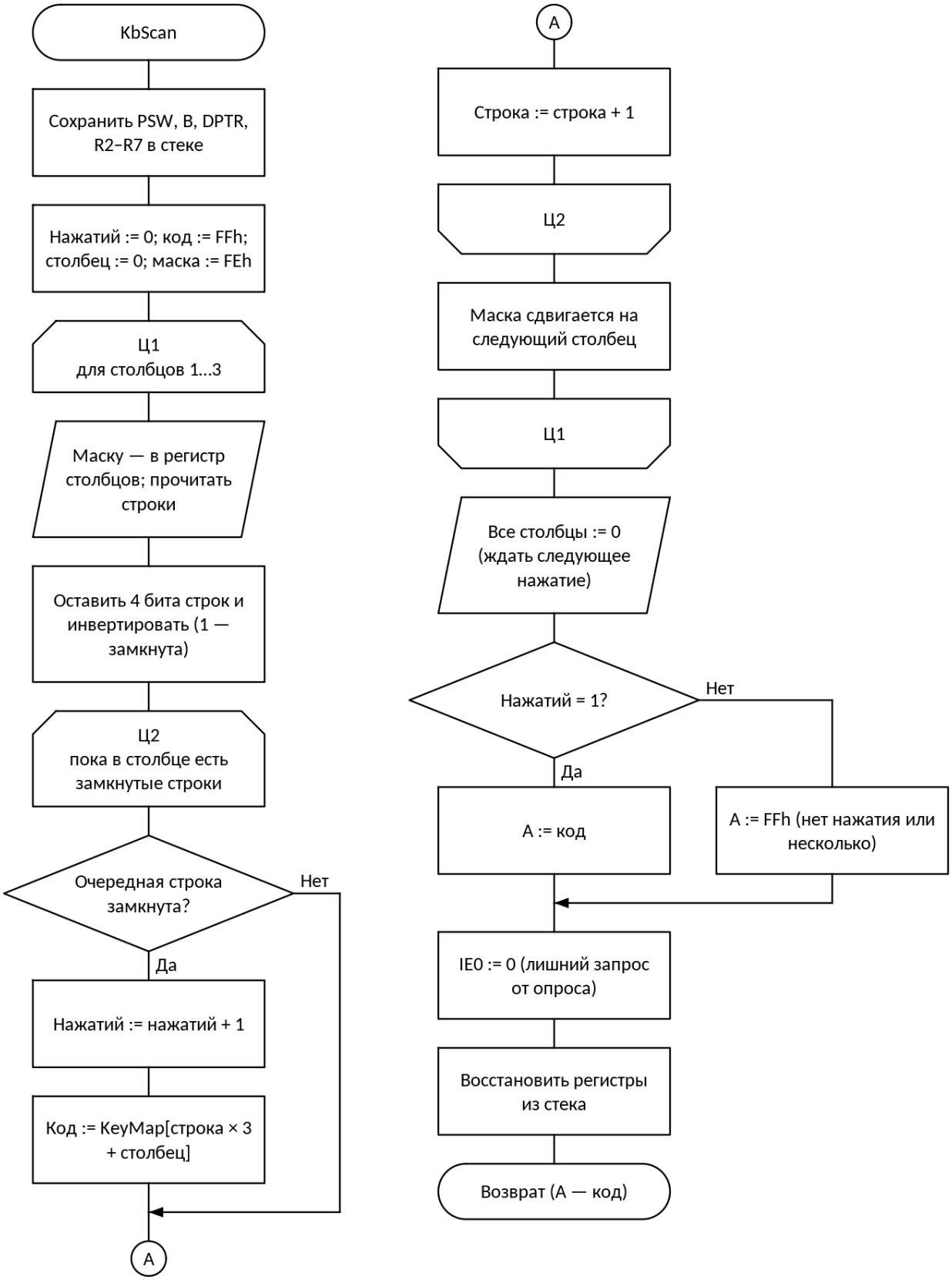


Рисунок 3 — Схема алгоритма: Опрос клавиатуры KbScan (обработчик INT0)

Процедура KbScan вызывается из обработчика INT0 и опрашивает клавиатуру 3×4 по столбцам. Сначала она сохраняет в стеке все регистры, которые будет портить, обнуляет счётчик нажатий и готовит маску FEh — ноль только в первом столбце. Для каждого столбца маска записывается в регистр столбцов, затем читаются строки; старшие биты шины отбрасываются, а оставшиеся четыре инвертируются, чтобы единица означала замкнутый

контакт. Во внутреннем цикле каждая замкнутая строка увеличивает счётчик и даёт код клавиши из таблицы KeyMap по номеру «строка × 3 + столбец». После перебора всех столбцов они снова опускаются в ноль, иначе следующее нажатие не вызовет прерывание. Если найдена ровно одна клавиша, в A возвращается её код, иначе FFh; флаг IE0 сбрасывается, потому что во время опроса линия INT0 сама несколько раз падала.

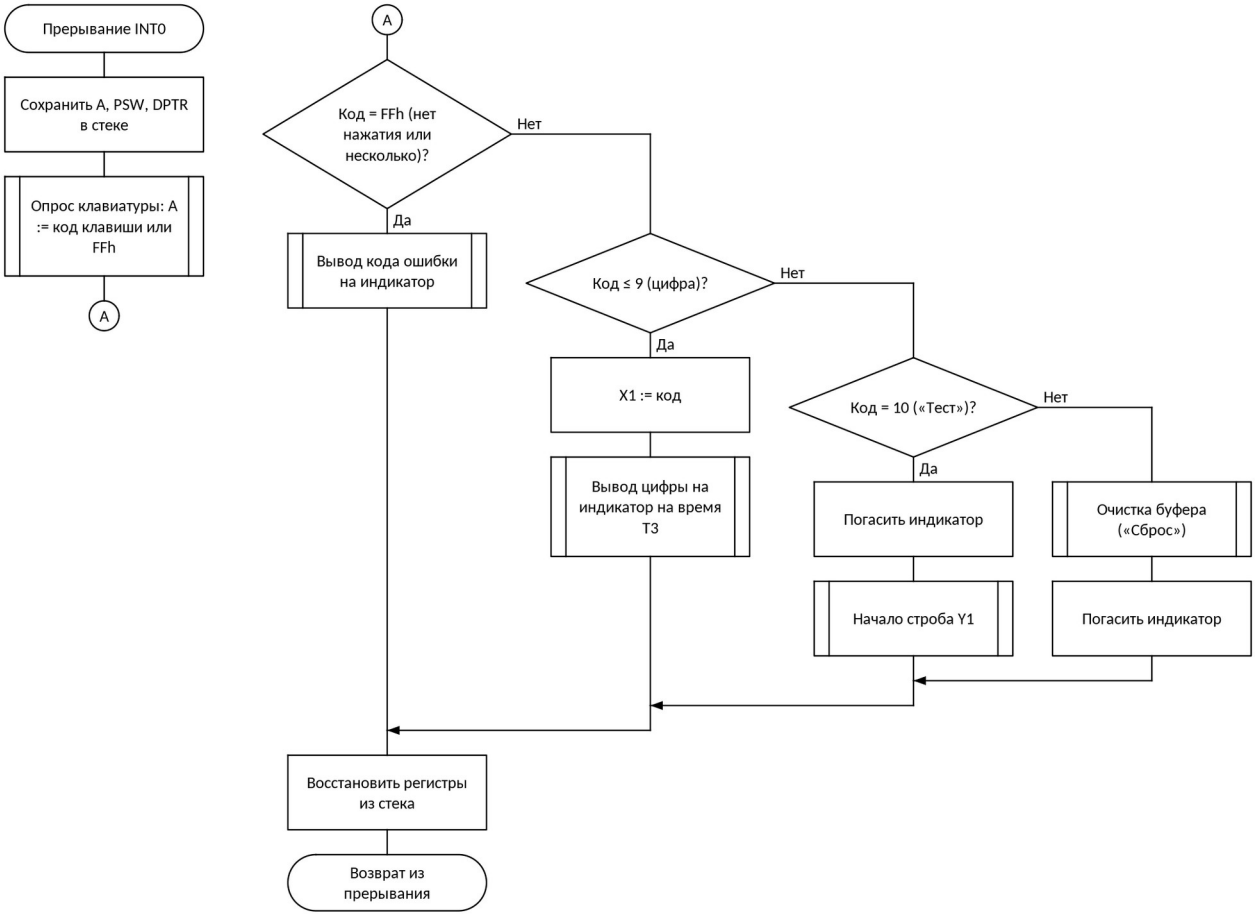


Рисунок 4 — Схема алгоритма: Обработчик прерывания INT0: реакция на клавишу

Схема «Обработчик прерывания INT0: реакция на клавишу» выполняется так: вход — «Прерывание INT0»; сохранить A, PSW, DPTR в стеке; вызывается подпрограмма: опрос клавиатуры: A := код клавиши или FFh; проверяется условие «Код = FFh (нет нажатия или несколько)»: если да — вызывается подпрограмма: вывод кода ошибки на индикатор, иначе — проверяется условие «Код ≤ 9 (цифра)»: если да — X1 := код, вызывается подпрограмма: вывод цифры на индикатор на время T3, иначе — проверяется условие «Код = 10 («Тест»)»: если да — погасить индикатор, вызывается подпрограмма: начало строба Y1, иначе — вызывается подпрограмма: очистка буфера («Сброс»), погасить индикатор; восстановить регистры из стека; в конце — возврат из прерывания.

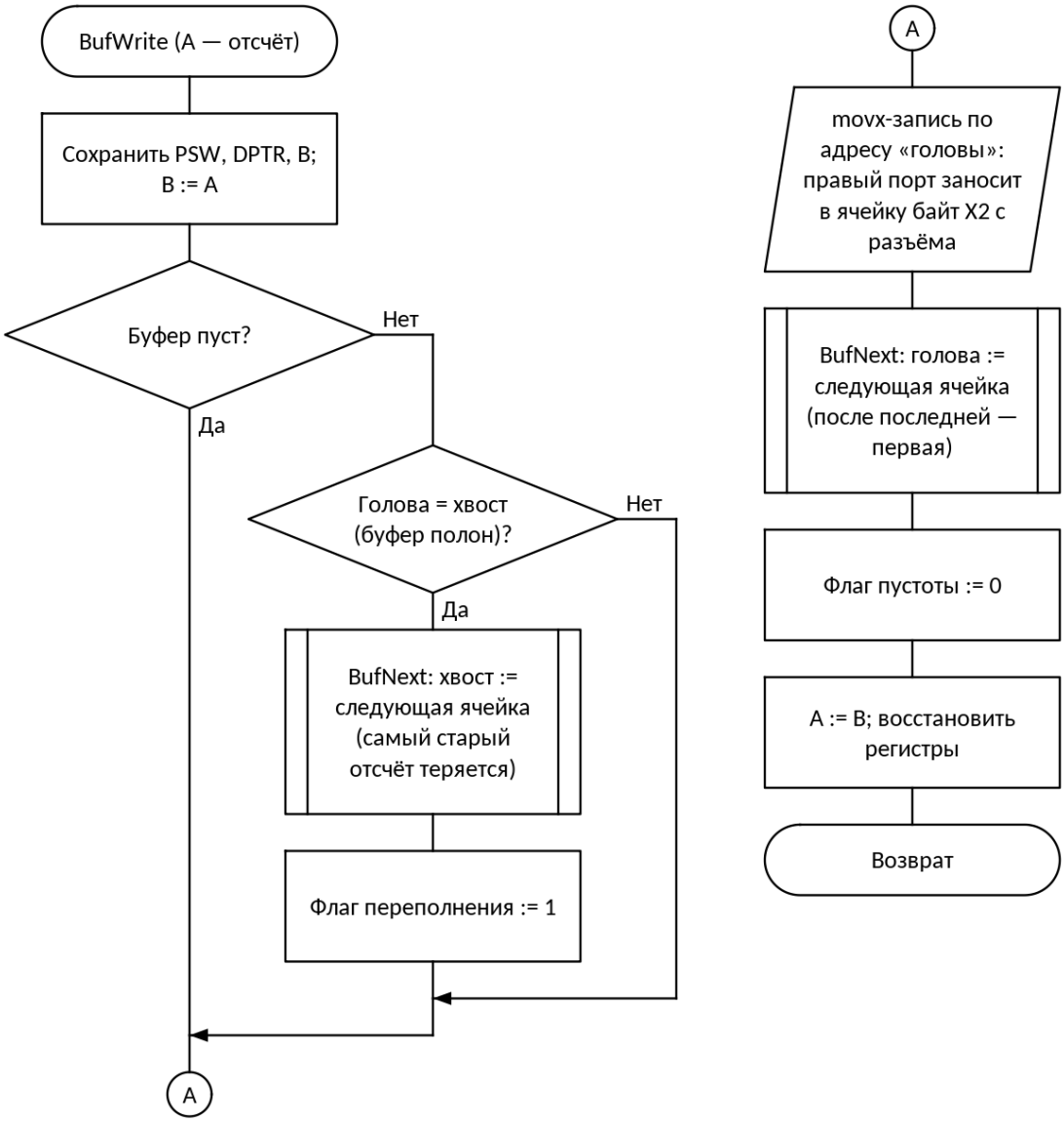


Рисунок 5 — Схема алгоритма: Запись отсчёта в кольцевой буфер BufWrite

Процедура BufWrite кладёт отсчёт из аккумулятора в ячейку, на которую указывает «голова». Перед записью она проверяет, не заполнен ли буфер целиком: это так, когда голова совпала с хвостом, а флаг пустоты сброшен. В этом случае самый старый отсчёт выбрасывается — хвост сдвигается на следующую ячейку, а флаг переполнения ставится в единицу. Потом отсчёт записывается в буфер, голова сдвигается вперёд, флаг пустоты сбрасывается. Сдвиг обоих указателей делает одна подпрограмма BufNext: она прибавляет единицу к адресу и после последней ячейки 0A6EDh возвращает указатель на начало 0A000h, сравнивая оба байта адреса.

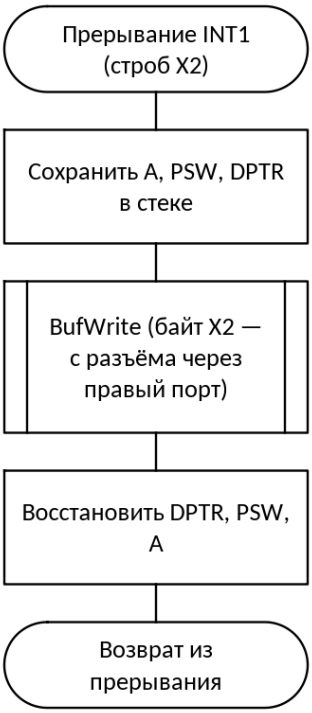


Рисунок 6 — Схема алгоритма: Обработчик прерывания INT1: приём отсчёта X2 от внешнего устройства

Схема «Обработчик прерывания INT1: приём отсчёта X2 от внешнего устройства» выполняется так: вход — «Прерывание INT1 (строб X2)»; сохранить A, PSW, DPTR в стеке; вызывается подпрограмма: BufWrite (байт X2 — с разъёма через правый порт); восстановить DPTR, PSW, A; в конце — возврат из прерывания.

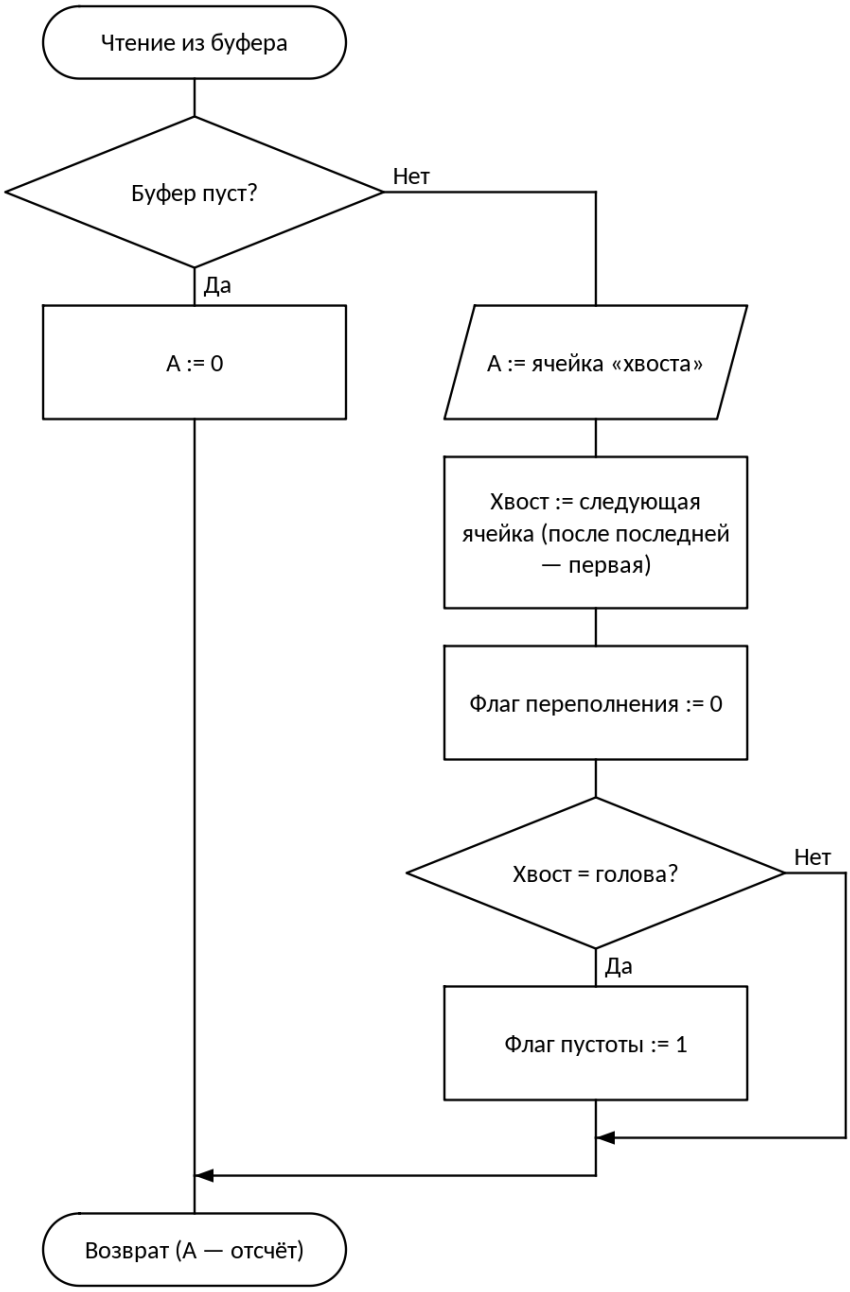


Рисунок 7 — Схема алгоритма: Чтение отсчёта из кольцевого буфера

Схема «Чтение отсчёта из кольцевого буфера» выполняется так: вход — «Чтение из буфера»; проверяется условие «Буфер пуст»: если да — A := 0, иначе — обмен с устройством: A := ячейка «хвоста», хвост := следующая ячейка (после последней — первая), флаг переполнения := 0, проверяется условие «Хвост = голова»: если да — флаг пустоты := 1; в конце — возврат (A — отсчёт).



Рисунок 8 — Схема алгоритма: Очистка кольцевого буфера (клавиша «Сброс»)

Схема «Очистка кольцевого буфера (клавиша «Сброс»)» выполняется так: вход — «Очистка буфера»; запретить прерывание INT1; хвост := голова; флаг пустоты := 1; флаг переполнения := 0; разрешить прерывание INT1; в конце — возврат.

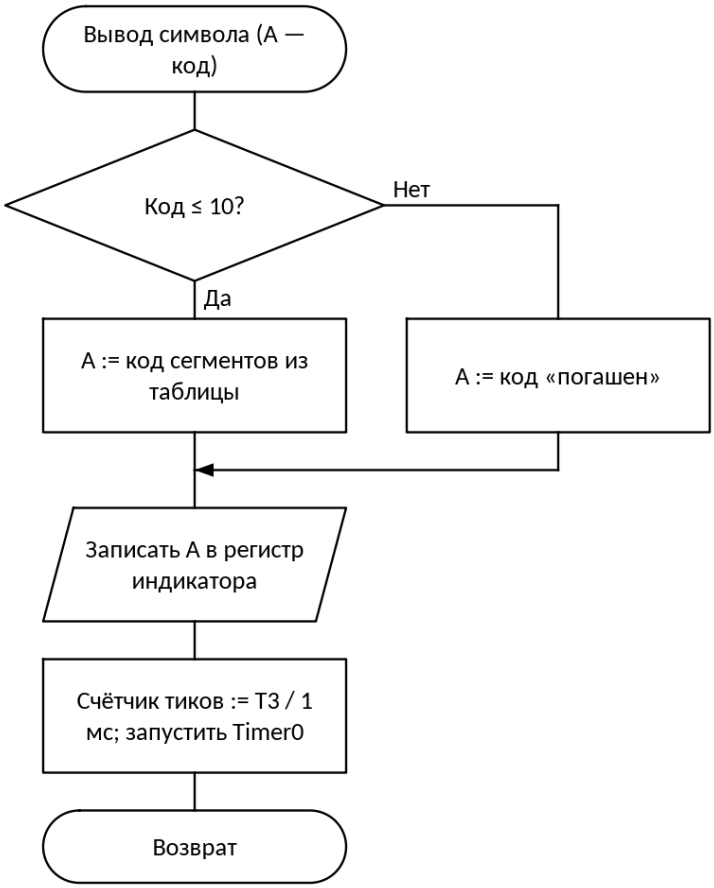


Рисунок 9 — Схема алгоритма: Вывод символа на индикатор

Схема «Вывод символа на индикатор» выполняется так: вход — «Вывод символа (A — код)»; проверяется условие «Код ≤ 10»: если да — A := код сегментов из таблицы, иначе — A := код «погашен»; обмен с устройством: записать A в регистр индикатора; счётчик тиков := T3 / 1 мс; запустить Timer0; в конце — возврат.

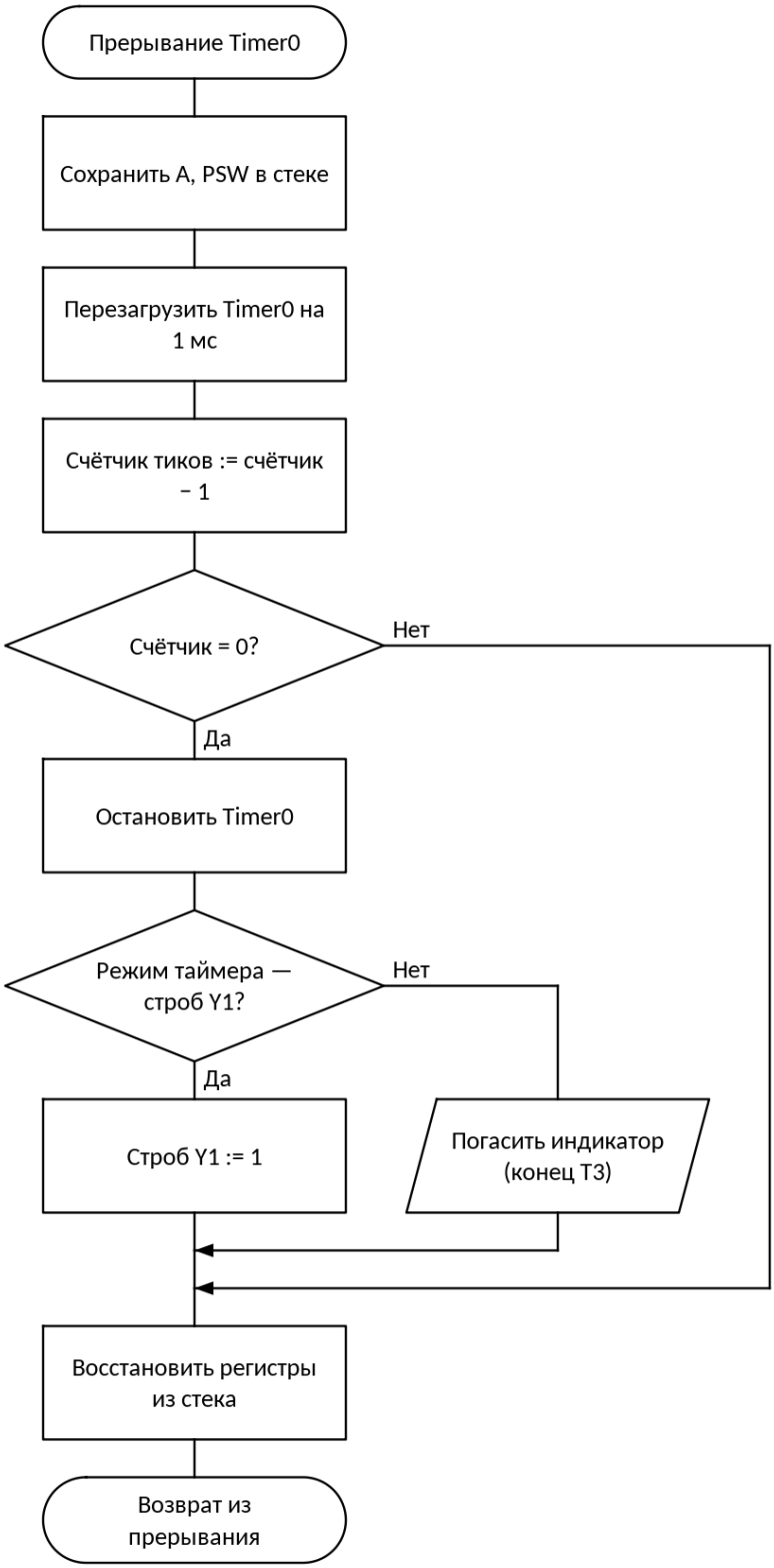


Рисунок 10 — Схема алгоритма: Обработчик прерывания Timer0: отсчёт T1 и T3

Схема «Обработчик прерывания Timer0: отсчёт T1 и T3» выполняется так: вход — «Прерывание Timer0»; сохранить A, PSW в стеке; перезагрузить Timer0 на 1 мс; счётчик тиков := счётчик − 1; проверяется условие «Счётчик = 0»: если да — остановить Timer0, проверяется условие «Режим таймера — строб Y1»: если да — строб Y1 := 1, иначе — обмен с устройством: погасить индикатор (конец T3); восстановить регистры из стека; в конце — возврат из прерывания.



Рисунок 11 — Схема алгоритма: Начало строба Y1

Схема «Начало строба Y1» выполняется так: вход — «Строб Y1»; счётчик тиков := T1 / 1 мс; Timer0 := тик 1 мс; строб Y1 := 0; запустить Timer0; в конце — возврат.

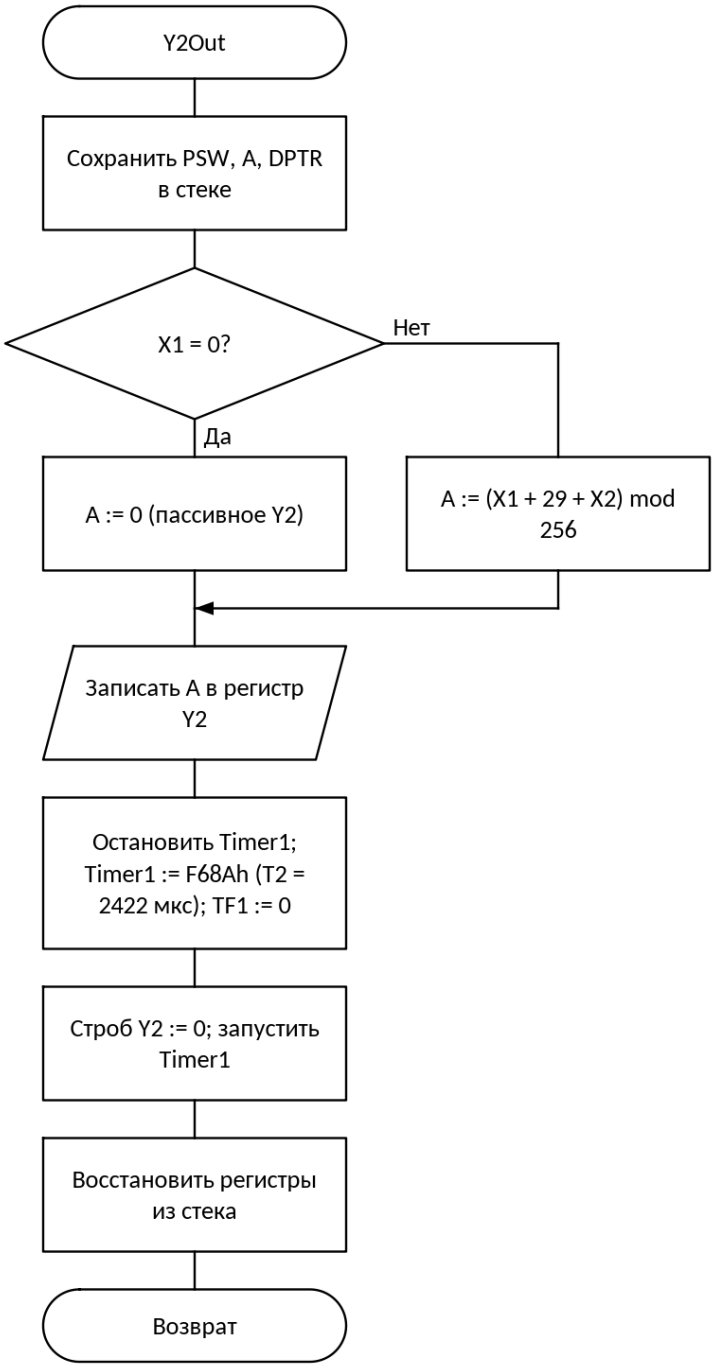


Рисунок 12 — Схема алгоритма: Расчёт и выдача сигнала Y2 (Y2Out)

Процедура Y2Out считает выходной сигнал и выдаёт его внешнему устройству. Если X1 равен нулю, Y2 пассивен и в регистр пишется ноль; иначе к X1 прибавляются константа G + M = 29 и отсчёт X2, а перенос просто отбрасывается — это и есть взятие по модулю 256. Значение сначала записывается в регистр Y2, и только после этого запускается строб, чтобы к его началу данные уже стояли на выходе регистра. Timer1 загружается значением F68Ah, его флаг сбрасывается, строб на P1.4 опускается в ноль и таймер запускается; процедура сразу возвращается, не дожидаясь конца строба.



Рисунок 13 — Схема алгоритма: Обработчик прерывания Timer1: конец строба Y2

Обработчик Timer1 срабатывает, когда таймер отсчитал T2 = 2422 мкс с начала строба. Он останавливает таймер, чтобы тот не дал повторного прерывания, и поднимает вывод P1.4 в единицу — строб Y2 закончен. Регистры обработчик не портит, поэтому ничего не сохраняет. Прерыванию Timer1 дан высокий приоритет, чтобы конец строба не задерживался, если в этот момент обрабатывается клавиатура или приём X2.

3.2 Структура программы с описанием функций составных частей и связи между ними

Программа состоит из инициализации, основной программы (ожидание прерываний) и подпрограмм, описанных ниже. Описание каждой подпрограммы взято из заголовочного комментария в исходном тексте (приложение А).

3.2.1 Подпрограмма KbScan (программа 1)

Назначение: опрос матрицы клавиатуры 3x4 по столбцам (вызывается из обработчика INT0). Столбцы опускаются по одному, в каждом считаются замкнутые строки; код берётся из таблицы KeyMap.

Входные данные: нет.

Выходные данные: A — код единственной нажатой клавиши (0…9, T = 10, C = 11) или 0FFh, если не нажато ничего или нажато больше одной клавиши.

Используемые регистры: только A (остальное сохраняется в стеке).

3.2.2 Подпрограмма BufWrite (программа 2)

Назначение: запись отсчёта в кольцевой буфер в ячейку «головы». Если буфер заполнен целиком (голова = хвост и буфер не пуст), самый старый отсчёт выбрасывается: хвост сдвигается на следующую ячейку и ставится флаг переполнения.

Входные данные: A — отсчёт.

Выходные данные: отсчёт в буфере, голова сдвинута, F_EMPTY = 0; F_OVF = 1, если отсчёт затёр самый старый.

Используемые регистры: ничего (A, B, DPTR, PSW сохраняются).

3.2.3 Подпрограмма BufNext (программа 2)

Назначение: следующая ячейка кольцевого буфера: после последней (BUF_END - 1) идёт первая (BUF_START).

Входные данные: DPTR — адрес ячейки буфера.

Выходные данные: DPTR — адрес следующей ячейки.

Используемые регистры: A, флаг C.

3.2.4 Подпрограмма Y2Out (программа 3)

Назначение: вычисление выхода Y2 и его выдача внешнему устройству. Y2 = (G + M + X1 + X2) mod 256, а при X1 = 0 — просто 0. Значение пишется в регистр Y2, после этого на P1.4 выдаётся строб 0 длиной T2; конец строба делает обработчик Timer1.

Входные данные: X1, X2.

Выходные данные: регистр Y2, строб на PIN_Y2 запущен.

Используемые регистры: ничего (A, DPTR, PSW сохраняются).

3.2.5 Обработчик прерывания Timer1 (программа 3)

Назначение: Timer1 — отсчитали T2, строб Y2 заканчивается.

Распределение ресурсов микроконтроллера — прерывания, адреса внешних устройств и переменные — приведено в таблицах 1–3.

Таблица 1 — Прерывания

Источник	**Вектор**	**Назначение**	**Запуск**
INT0 (P3.2)	0003h	нажатие клавиши	по спаду
Timer0	000Bh	отсчёт T_1 (361 мс) и T_3 (3190 мс), тик 1 мс	переполнение
INT1 (P3.3)	0013h	строб записи X_2	по спаду
Timer1	001Bh	отсчёт T_2 (2422 мкс)	переполнение
UART	0023h	не используется (заглушка)	—

Таблица 2 — Адреса внешних устройств (обращение командами movx при P3.4 = 1)

Устройство	**Выход дешифратора**	**Адреса**
Индикатор	Y6	0C000h–0DFFFh
Клавиатура	Y1	2000h–3FFFh
Буфер (IDT7005)	Y5	0A000h–0BFFFh
Регистр Y2	Y7	0E000h–0FFFFh

Таблица 3 — Переменные во внутреннем ОЗУ

Имя	**Адрес**	**Назначение**
HEAD_L, HEAD_H	47h, 48h	«голова» — адрес ячейки буфера для следующей записи
TAIL_L, TAIL_H	49h, 4Ah	«хвост» — адрес ячейки буфера для следующего чтения
F_EMPTY	22h.1 (бит 11h)	флаг пустоты буфера
F_OVF	23h.5 (бит 1Dh)	флаг переполнения буфера
X1	30h	сигнал X_1 — цифра с клавиатуры
X2	31h	сигнал X_2 — отсчёт из буфера

3.3 Связи программы с другими программами

Программа «Программа управления объектом» в ходе своей работы не использует других программ.

4 Входные данные

4.1 Клавиатура

Клавиатура — матрица 3×4 (3 столбцов × 4 строк), сканирование по столбцам: в регистр столбцов (адрес 2000h, разряды D0…D2) выдаётся 0 в опрашиваемый столбец, строки читаются через повторитель по тому же адресу (разряды D0…D3, 0 — контакт замкнут). Нажатие любой клавиши при всех столбцах в 0 вызывает прерывание INT0. Коды клавиш (табл. 3 ТЗ) и их расположение — в таблице 4.

Таблица 4 — Расположение клавиш и их коды («клавиша» — код)

	Col1	**Col2**	**Col3**
Row1	«1» — 1	«2» — 2	«3» — 3
Row2	«4» — 4	«5» — 5	«6» — 6
Row3	«7» — 7	«8» — 8	«9» — 9
Row4	«0» — 0	«Т» — 10	«С» — 11

Результат опроса — код единственной нажатой клавиши (0–9 — цифры, 10 — «Тест», 11 — «Сброс») или 0FFh, если не нажато ничего или нажато больше одной клавиши. Цифра становится значением X_1 (начальное значение $X_1 = 0$).

4.2 Сигнал X2

X_2 — 8-разрядное целое без знака. Внешнее устройство выставляет отсчёт на разъёме доступа к буферу и сопровождает его стробом, который вызывает прерывание INT1. Буфером управляет МК: обработчик делает movx-запись по адресу «головы», и правый порт двухпортового ОЗУ IDT7005 заносит в ячейку байт X_2 с разъёма. Отсчёты хранятся в кольцевом буфере на 1774 байт (адреса 0A000h–0A6EDh).

5 Выходные данные

5.1 Характер и организация выходных данных

Индикация. На индикаторе отображается последняя введённая цифра — до ввода следующей, но не дольше T_3 = 3,190 с; при ошибке — код ошибки с горящей точкой dp. Начальное состояние — индикатор погашен.

Кольцевой буфер во внешнем ОЗУ. Ячейки 0A000h–0A6EDh. «Голова» — адрес ячейки для следующей записи, «хвост» — для следующего чтения; после последней ячейки следует первая. Буфер пуст — флаг пустоты = 1 (голова = хвост). Если буфер заполнен, новый отсчёт затирает самый старый (хвост сдвигается) и устанавливается флаг переполнения; после чтения флаг переполнения сбрасывается.

Управляющие сигналы. Y_1 — строб низкого уровня длительностью 361 мс по клавише «Тест». $Y_2 = (12 + 17 + X_1 + X_2) \bmod 256$ при $X_1 \neq 0$, иначе $Y_2 = 0$ (пассивное состояние); вырабатывается, пока в буфере есть данные и $X_1 \neq 0$. Каждая запись Y_2 в выходной регистр сопровождается стробом длительностью 2422 мкс; до записи регистр выдаёт предыдущее значение.

Расчётные параметры работы программы. Константы таймеров (кварц 12 МГц, машинный цикл $T_{мц}$ = 12 / f = 1 мкс, режим 1 — 16 бит):

TH1:TL1 = 65536 − T2 / T_мц = 65536 − 2422 = 63114 = 0F68Ah (строб Y2, таймер 1)

TH0:TL0 = 65536 − 1 мс / T_мц = 65536 − 1000 = 64536 = 0FC18h (тик 1 мс, таймер 0)

N_T1 = T1 / 1 мс = 361; N_T3 = T3 / 1 мс = 3190 (число тиков)

Минимальное время заполнения буфера (при $X_1 = 0$ сигнал Y_2 пассивен и буфер не читается, отсчёты поступают с максимальной интенсивностью):

t_min = V / f_X2 = 1774 / 4 = 443,5 с (7,4 мин).

Максимальная глубина стека для программы 1 — 16 байт (SP от 07h до 17h). Значение получено прогоном программы в симуляторе на всех сценариях проверки (каждая клавиша, одновременные нажатия, помеха на INT0) и складывается из адреса возврата при входе в обработчик INT0 (2 байта), адреса возврата при вызове процедуры опроса (2 байта) и сохраняемых в стеке регистров.

5.2 Формат, описание и способ кодирования выходных данных

Код символа (табл. 2 ТЗ) преобразуется в код сегментов по таблице 5: разряды D0…D7 регистра индикатора соответствуют сегментам a, b, c, d, e, f, g, dp; индикатор с общим анодом — сегмент горит при 0.

Таблица 5 — Коды сегментов индикатора (dp g f e d c b a)

Код	**Символ**	**Двоичный**	**Шестн.**
0	«0»	11000000	0C0h
1	«1»	11111001	0F9h
2	«2»	10100100	0A4h
3	«3»	10110000	0B0h
4	«4»	10011001	99h
5	«5»	10010010	92h
6	«6»	10000010	82h
7	«7»	11111000	0F8h
8	«8»	10000000	80h
9	«9»	10010000	90h
10	«E»	10000110	86h
11 и больше	погашен	11111111	0FFh

Таблица 6 — Коды ошибок (символ с горящей точкой dp, индикация до нажатия любой клавиши, но не дольше Т3)

Индикация	**Смысл**	**Когда выставляется**
«1.»	нажато несколько клавиш	опрос клавиатуры нашёл больше одного замкнутого контакта
«2.»	нажатие не распознано	прерывание пришло, а замкнутых контактов нет
«3.»	переполнение буфера	запись X_2 затёрла непрочитанный отсчёт
«4.»	повторный «Тест»	«Тест» нажат, пока идёт строб Y_1

Таблица описывает полную систему. В моих трёх программах ошибки 1 и 2 процедура KbScan сообщает одним кодом FFh (несколько клавиш или ни одной), а различать их и выводить символ на индикатор должен обработчик INT0 (рисунок 4). Ошибку 3 отмечает процедура BufWrite: при записи в заполненный буфер она ставит флаг переполнения. Ошибка 4 относится к стробу Y1, который в мои программы не входит.

Сигнал Y_2 выдаётся 8-разрядным кодом в выходной регистр (адрес 0E000h), строб — на выводе P1.4; сигнал Y_1 — на выводе P1.3.

В 16-разрядном режиме (режим 1) таймер 0 перезагружается в обработчике прерывания; задержка входа в прерывание (несколько машинных циклов на тик) компенсируется поправкой к загружаемому значению или учитывается как погрешность длительности. В режиме 2 (8 разрядов с автоперезагрузкой) перезагрузку выполняет сам таймер, а тик набирается программным счётчиком переполнений.

ПЕРЕЧЕНЬ ПРИНЯТЫХ СОКРАЩЕНИЙ

Таблица 7 — Сокращения

Сокращение	**Расшифровка**
МК	микроконтроллер
МПС	микропроцессорная система
ОЗУ	оперативное запоминающее устройство
ПЗУ	постоянное запоминающее устройство
ЕСПД	единая система программной документации
ТЗ	техническое задание
CS	Chip Select, выбор микросхемы
IRQ	запрос прерывания
SP	указатель стека
DPTR	16-разрядный регистр-указатель данных
dp	десятичная точка индикатора

ЛИСТ РЕГИСТРАЦИИ ИЗМЕНЕНИЙ

Изм.	Номера листов (страниц)				Всего листов (страниц) в докум.	№ докум.	Входящий № сопроводит. докум. и дата	Подп.	Дата
	изменённых	заменённых	новых	аннулиро-ванных					

ПРИЛОЖЕНИЕ А

Листинги программ

Программа 1 — файл «Рязанцев ИВ-код-1»

```
; Рязанцев И.В., А-12-23, 17, v1
$INCLUDE (vars.inc)             ; адреса, биты, константы варианта; в файл для робота mpscode вклеит его текст
DSEG AT 30h
; переменные не нужны: всё держим в регистрах банка 0
CSEG AT 0h
        sjmp START
org 03h                         ; INT0 — нажатие клавиши: процедура вызывается отсюда
        call KbScan ; %proc%
        reti
org 0bh ; "заглушка" для Tmr0_ovf
        nop
        reti
org 13h ; "заглушка" для INT1
        nop
        reti
org 1bh ; "заглушка" для Tmr1_ovf
        nop
        reti
org 23h ; "заглушка" для UART
        nop
        reti
org 2bh
START:
; Инициализация МК ******************************************
        mov SP, #07h
        setb PIN_CSEN           ; разрешаем дешифратор, иначе клавиатура не выбирается
        mov DPTR, #ADR_KB
        clr A
        movx @DPTR, A           ; опускаем все столбцы: любое нажатие сразу опустит INT0
        setb IT0                ; INT0 срабатывает по спаду
        clr IE0                 ; забываем возможный ложный запрос до инициализации
        setb EX0                ; разрешаем прерывание от клавиатуры
        setb EA                 ; и прерывания вообще
        jmp $ ; %stop%

; KbScan — опрос матрицы клавиатуры 3x4 по столбцам (вызывается из обработчика INT0).
; Столбцы опускаются по одному, в каждом считаются замкнутые строки; код берётся из таблицы KeyMap.
; Вход: нет. Выход: A — код единственной нажатой клавиши (0…9, T = 10, C = 11) или 0FFh,
; если не нажато ничего или нажато больше одной клавиши. Портит: только A (остальное сохраняется в стеке).
KbScan:
        push PSW
        push B
        push DPL
        push DPH
        push AR2
        push AR3
        push AR4
        push AR5
        push AR6
        push AR7
        mov DPTR, #ADR_KB
        mov R2, #0              ; сколько замкнутых контактов нашли
        mov R3, #0FFh           ; код последней найденной клавиши
        mov R4, #0              ; номер текущего столбца (0…KB_COLS-1)
        mov R5, #0FEh           ; маска: опущен только первый столбец
KsCol:
        mov A, R5
        movx @DPTR, A           ; опускаем текущий столбец, остальные подняты
        movx A, @DPTR           ; читаем строки
        anl A, #(1 SHL KB_ROWS) - 1     ; старшие биты шины не подключены
        xrl A, #(1 SHL KB_ROWS) - 1     ; теперь 1 — строка замкнута на этот столбец
        mov R7, A               ; замкнутые строки этого столбца
        mov R6, #0              ; номер строки
KsRow:
        mov A, R7
        jz KsNextCol            ; в этом столбце замкнутых строк больше нет
        clr C
        rrc A                   ; младшая строка уходит в перенос
        mov R7, A
        jnc KsRowSkip
        inc R2                  ; нашли ещё одну нажатую клавишу
        mov A, R6               ; индекс в таблице = строка * KB_COLS + столбец
        mov B, #KB_COLS
        mul AB
        add A, R4
        push DPL                ; адрес клавиатуры ещё понадобится
        push DPH
        mov DPTR, #KeyMap
        movc A, @A+DPTR         ; код клавиши по рис. 4 и табл. 3 ТЗ
        mov R3, A
        pop DPH
        pop DPL
```

```
KsRowSkip:
        inc R6
        sjmp KsRow
KsNextCol:
        mov A, R5
        rl A                    ; опускать будем следующий столбец
        mov R5, A
        inc R4
        cjne R4, #KB_COLS, KsCol
        clr A
        movx @DPTR, A           ; опрос закончен: все столбцы снова вниз, чтобы ловить следующее нажатие
        mov A, R3
        cjne R2, #1, KsBad      ; ровно одна клавиша — отдаём её код
        sjmp KsDone
KsBad:
        mov A, #0FFh            ; ничего или несколько клавиш сразу — ошибка
KsDone:
        clr IE0                 ; пока перебирали столбцы, INT0 дёргался — этот запрос лишний
        pop AR7
        pop AR6
        pop AR5
        pop AR4
        pop AR3
        pop AR2
        pop DPH
        pop DPL
        pop B
        pop PSW
        ret

; KeyMap — коды клавиш по строкам сверху вниз, в строке — по столбцам слева направо (рис. 4 ТЗ, 3x4).
KeyMap: DB 1, 2, 3
        DB 4, 5, 6
        DB 7, 8, 9
        DB 0, 10, 11
END
```

Программа 2 — файл «Рязанцев ИВ-код-2»

```
; Рязанцев И.В., А-12-23, 17, v1
$INCLUDE (vars.inc)             ; адреса, биты, константы варианта; в файл для робота mpscode вклеит его текст
DSEG AT 30h
; переменные не нужны: указатели и флаги заданы в vars.inc
CSEG AT 0h
        sjmp START
org 03h ; "заглушка" для INT0
        nop
        reti
org 0bh ; "заглушка" для Tmr0_ovf
        nop
        reti
org 13h ; "заглушка" для INT1
        nop
        reti
org 1bh ; "заглушка" для Tmr1_ovf
        nop
        reti
org 23h ; "заглушка" для UART
        nop
        reti
org 2bh
START:
; Инициализация МК ********************************************
        mov SP, #07h
        setb PIN_CSEN           ; дешифратор включён — буфер доступен по movx
        mov HEAD_L, #LOW(BUF_START)     ; голова — в начало буфера
        mov HEAD_H, #HIGH(BUF_START)
        mov TAIL_L, #LOW(BUF_START)     ; хвост — туда же: буфер пуст
        mov TAIL_H, #HIGH(BUF_START)
        setb F_EMPTY            ; данных нет
        clr F_OVF               ; ничего не терялось
        mov A, #55h             ; пример отсчёта
        call BufWrite ; %proc%
        jmp $ ; %stop%

; BufWrite — запись отсчёта в кольцевой буфер в ячейку «головы».
; Если буфер заполнен целиком (голова = хвост и буфер не пуст), самый старый отсчёт выбрасывается:
; хвост сдвигается на следующую ячейку и ставится флаг переполнения.
; Вход: A — отсчёт. Выход: отсчёт в буфере, голова сдвинута, F_EMPTY = 0; F_OVF = 1, если отсчёт затёр самый старый.
; Портит: ничего (A, B, DPTR, PSW сохраняются).
BufWrite:
        push PSW                ; флаги вызывающей программы не трогаем
        push DPL                ; DPTR тоже сохраняем — он нужен для movx
        push DPH
        push B                  ; B будет временным местом для отсчёта
        mov B, A                ; отсчёт пока отложим в B
        jb F_EMPTY, BwStore     ; пустой буфер не может быть полным
        mov A, HEAD_L           ; полон ли: голова догнала хвост?
        cjne A, TAIL_L, BwStore
        mov A, HEAD_H
        cjne A, TAIL_H, BwStore
```

```
        mov DPL, TAIL_L         ; полон — выбрасываем самый старый отсчёт
        mov DPH, TAIL_H
        call BufNext
        mov TAIL_L, DPL
        mov TAIL_H, DPH
        setb F_OVF              ; отметили потерю данных
BwStore:
        mov DPL, HEAD_L
        mov DPH, HEAD_H
        mov A, B
        movx @DPTR, A           ; кладём отсчёт в ячейку головы
        call BufNext
        mov HEAD_L, DPL         ; голова смотрит на следующую свободную ячейку
        mov HEAD_H, DPH
        clr F_EMPTY             ; теперь в буфере точно что-то есть
        mov A, B                ; вернули отсчёт в A
        pop B                   ; восстанавливаем всё, что сохранили, в обратном порядке
        pop DPH
        pop DPL
        pop PSW
        ret                     ; запись закончена

; BufNext — следующая ячейка кольцевого буфера: после последней (BUF_END - 1) идёт первая (BUF_START).
; Вход: DPTR — адрес ячейки буфера. Выход: DPTR — адрес следующей ячейки. Портит: A, флаг C.
BufNext:
        inc DPTR
        mov A, DPL
        cjne A, #LOW(BUF_END), BnDone   ; ещё не вышли за конец — готово
        mov A, DPH
        cjne A, #HIGH(BUF_END), BnDone
        mov DPTR, #BUF_START    ; вышли за конец — заворачиваем на начало
BnDone:
        ret                     ; DPTR — следующая ячейка
END
```

Программа 3 — файл «Рязанцев ИВ-код-3»

```
; Рязанцев И.В., А-12-23, 17, v1
$INCLUDE (vars.inc)             ; адреса, биты, константы варианта; в файл для робота mpscode вклеит его текст
DSEG AT 30h
X1:     DS 1                    ; цифра с клавиатуры (0 — Y2 пассивен)
X2:     DS 1                    ; отсчёт из буфера
CSEG AT 0h
        sjmp START
org 03h ; "заглушка" для INT0
        nop
        reti
org 0bh ; "заглушка" для Tmr0_ovf
        nop
        reti
org 13h ; "заглушка" для INT1
        nop
        reti
org 1bh                         ; Timer1 — отсчитали T2, строб Y2 заканчивается
        clr TR1                 ; таймер больше не нужен
        setb PIN_Y2             ; строб в пассивную единицу
        reti
org 23h ; "заглушка" для UART
        nop
        reti
org 2bh
START:
; Инициализация МК ********************************************
        mov SP, #07h
        setb PIN_CSEN           ; дешифратор включён — регистр Y2 доступен по movx
        setb PIN_Y2             ; строб Y2 пока не активен
        anl TMOD, #0Fh          ; Timer1: режим 1 (16 бит), Timer0 не трогаем
        orl TMOD, #10h
        setb PT1                ; конец строба важнее остального — высокий приоритет
        setb ET1                ; прерывание от Timer1
        setb EA
        mov X1, #3              ; пример входных данных
        mov X2, #200
        call Y2Out ; %proc%
        jmp $ ; %stop%

; Y2Out — вычисление выхода Y2 и его выдача внешнему устройству.
; Y2 = (G + M + X1 + X2) mod 256, а при X1 = 0 — просто 0. Значение пишется в регистр Y2,
; после этого на P1.4 выдаётся строб 0 длиной T2; конец строба делает обработчик Timer1.
; Вход: X1, X2. Выход: регистр Y2, строб на PIN_Y2 запущен. Портит: ничего (A, DPTR, PSW сохраняются).
Y2Out:
        push PSW
        push ACC
        push DPL
        push DPH
        mov A, X1
        jz YoWrite              ; X1 = 0 — выдаём ноль
        add A, #Y2_CONST        ; G + M + X1; перенос отбрасывается — это и есть mod 256
        add A, X2
YoWrite:
        mov DPTR, #ADR_Y2
```

```
        movx @DPTR, A           ; сначала данные в регистр
        clr TR1                 ; на случай, если прошлый строб ещё идёт — перезапускаем отсчёт
        mov TH1, #T2_RELOAD_H   ; Timer1 отсчитает T2
        mov TL1, #T2_RELOAD_L
        clr TF1                 ; старый запрос не нужен
        clr PIN_Y2              ; строб пошёл
        setb TR1
        pop DPH
        pop DPL
        pop ACC
        pop PSW
        ret
END
```